



Tecnológico de Monterrey

Evidencia 2. Avances y presentación del reto

Integrantes del equipo:

Itzel Covarrubias Basurto | A01384322

Dariana Vega Morales | A01425755

Karla DeLa Cruz Valencia | A01738420

Oriana Isabella Cañizales Hernandez | A01738027

Profesor:

Iván Olmos Pineda

Luciano García Bañuelos

Materia:

Modelación de sistemas multiagentes con gráficas computacionales

(Gpo 501)

30 de noviembre del 2025

1. Escenario del Mini-Juego

El proyecto consiste en un mini-juego tipo Escape Room medieval en donde el usuario controla a un personaje dentro de una habitación cerrada. La misión del jugador es recolectar todas las llaves para desbloquear la puerta de salida.

La visualización es en tercera persona, esta perspectiva de cámara facilita la inmersión del usuario y optimiza la visibilidad del entorno y la interacción con los elementos del juego.

El renderizado gráfico y ambiental se realiza utilizando OpenGL implementado a través de Python.

2. Personajes y Comportamientos

2.1. Personaje principal (controlado por el usuario)

- Control mediante teclado (flechas de dirección).
- Animación de extremidades al caminar (brazos y piernas).
- Acción de recoger llave usando la tecla Espacio.
- Comportamiento realista con colisiones (no atraviesa paredes u objetos sólidos).

2.2. Fantasma (agente autónomo de Agents.jl)

- Actúa mediante lógica reactiva programada en Julia, utilizando el modelo pathfindig con A estrella.
- Identifica posiciones de las llaves gracias al estado mantenido en Agents.jl.
- Puede atravesar paredes (propio de su naturaleza).
- En el modo guía (avance 1), su objetivo es apoyar al jugador mostrando la ruta hacia las llaves.
- En la versión final, cuando el jugador toca la mesa, se activa el modo enemigo:
 - Compite por recolectar llaves antes que el jugador.
 - Se mueve estratégicamente hacia la llave más cercana.

- Si obtiene más llaves que el jugador, gana el juego.

2.3. Llaves (agentes del entorno)

- Su estado (recolectada) se mantiene en Julia.
- Desaparecen del modelo 3D una vez recolectadas.

2.4. Puerta

- Renderizada como parte del entorno.
- Permanece cerrada mientras existan llaves pendientes.
- Cuando se recolectan todas y el personaje principal (controlado por el usuario) se acerca a la puerta, se activa la animación de apertura que marca el final del juego.

3. Aspectos Gráficos de los Personajes

3.1. Modelado y Animación

- Todos los modelos 3D fueron elaborados o procesados en **Blender**.
- Se exportan en formato **.obj** y se renderizan en Python usando **OpenGL**.
- El personaje principal incluye:
 - ✓ animación básica de marcha
 - ✓ movimiento de extremidades
 - ✓ modelado de cuerpo completo

3.2. Movimiento fino

La simulación gráfica del juego se actualiza con una frecuencia mayor que la lógica interna de los agentes, lo que permite representar movimientos más suaves y naturales en pantalla. Aunque los cambios de estado en Agents.jl ocurren en intervalos más amplios, el motor

gráfico interpola visualmente entre estas actualizaciones, evitando desplazamientos bruscos y dando una sensación de continuidad en el movimiento del personaje y del fantasma.

4. Arquitectura del Software y Tecnologías utilizadas

El sistema se organiza como una arquitectura cliente–servidor local, donde **Python** se encarga del renderizado gráfico e interacción con el jugador, y **Julia** gestiona la simulación basada en agentes expuesta mediante un **API Web**.

- **Cliente gráfico (Python + PyGame + OpenGL)**

El archivo `main.py` implementa el bucle principal del juego usando PyGame para la creación de la ventana, el manejo del teclado y el renderizado de texto en pantalla. Los modelos 3D del personaje, el fantasma, las llaves, la puerta y el escenario se cargan mediante `objloader` y se dibujan con PyOpenGL (OpenGL.GL, OpenGL.GLU, OpenGL.GLUT). Desde este cliente también se calcula la lógica de colisión, la animación de las extremidades del personaje y la interfaz de usuario (mensajes contextuales).

- **Servidor de simulación (Julia + Agents.jl + Webapi)**

El archivo `webapi.jl` incluye el modelo definido en `agents.jl` y utiliza `Agents.jl` para representar al fantasma y a las llaves como agentes dentro de un entorno discreto. La simulación se ejecuta en Julia y se expone mediante un servidor HTTP implementado con `Webapi.jl`, que define rutas como `/init_keys`, `/trigger_evil`, `/collect_key` y `/update`. En cada llamada a `/update`, el servidor avanza la simulación con `step!(model, 1)` y devuelve el estado actualizado del fantasma y de las llaves en formato JSON.

- **Comunicación mediante API Web (HTTP + JSON)**

Desde `main.py`, el cliente en Python se comunica con el backend en Julia utilizando la librería `requests`, enviando peticiones HTTP a

`http://localhost:8000`. Se emplean peticiones POST para inicializar las llaves (`/init_keys`), notificar que el jugador tocó la mesa (`/trigger_evil`) o recogió una llave (`/collect_key`), y peticiones GET a `/update` para obtener la posición del fantasma, su estado (normal o “evil”) y las llaves visibles. Esta API sincroniza el estado lógico de Agents.jl con la representación visual en OpenGL, permitiendo que el fantasma tome decisiones autónomas mientras el jugador interactúa en tiempo real con el entorno 3D.

5. Reflexiones Finales

Itzel: De lo que más llevo de este proyecto fue entender la lógica que hay sobre la conexión entre Julia y OpenGL, las cuales no son herramientas comunes e hicieron salirme de mi zona de confort, entender que Python trabaja más rápido que Julia y saber llegar a ese intermedio para que ambos se pudieran ejecutar bien fue uno de los grandes retos que tuve. Me gustó mucho experimentar y aprender más sobre Blender y así exportar nuestro entorno a OpenGL, al igual que la animación de los personajes para que se vieran más naturales. Sin duda fue un proyecto de muchos aprendizajes y retos, pero interesante.

Ori: Este proyecto me permitió comprender cómo integrar correctamente distintos componentes de software —gráficos, simulación y comunicación por API— en un solo sistema funcional. Antes pensaba que cada tecnología operaba de forma aislada, pero trabajar con OpenGL, Agents.jl y un servidor en Julia me hizo entender la importancia de diseñar arquitecturas modulares y sincronizadas. La experiencia reforzó mi capacidad para dividir problemas complejos en partes manejables y para coordinar procesos independientes de manera robusta, algo esencial en el desarrollo de videojuegos y en la ingeniería de software en general.

Karla: La parte más valiosa del proyecto fue aprender a manejar agentes autónomos y ver cómo decisiones sencillas en la simulación pueden generar comportamientos complejos y realistas. Implementar la rivalidad entre el fantasma y el jugador me ayudó a visualizar cómo los modelos reactivos pueden adaptarse dinámicamente al entorno. Además, ver cómo el servidor de Julia y el cliente en Python se mantienen sincronizados me dio una visión clara de

cómo funcionan los sistemas distribuidos y de la importancia de definir protocolos de comunicación estables.

Dari: El proyecto me ayudó a fortalecer mi capacidad de trabajo colaborativo y mi comprensión del flujo completo de un videojuego: desde la creación de modelos 3D, hasta la simulación de agentes y la gestión de estados mediante un API. También aprendí la importancia de probar, depurar y ajustar constantemente los módulos para mantener la coherencia del sistema. Esta experiencia me dejó claro que la ingeniería de software no es solo programar, sino coordinar tecnologías, resolver problemas en equipo y construir sistemas que funcionen de manera confiable bajo condiciones reales.

6. Link del repositorio

<https://github.com/ItzelCovas/EscapeRoom>